

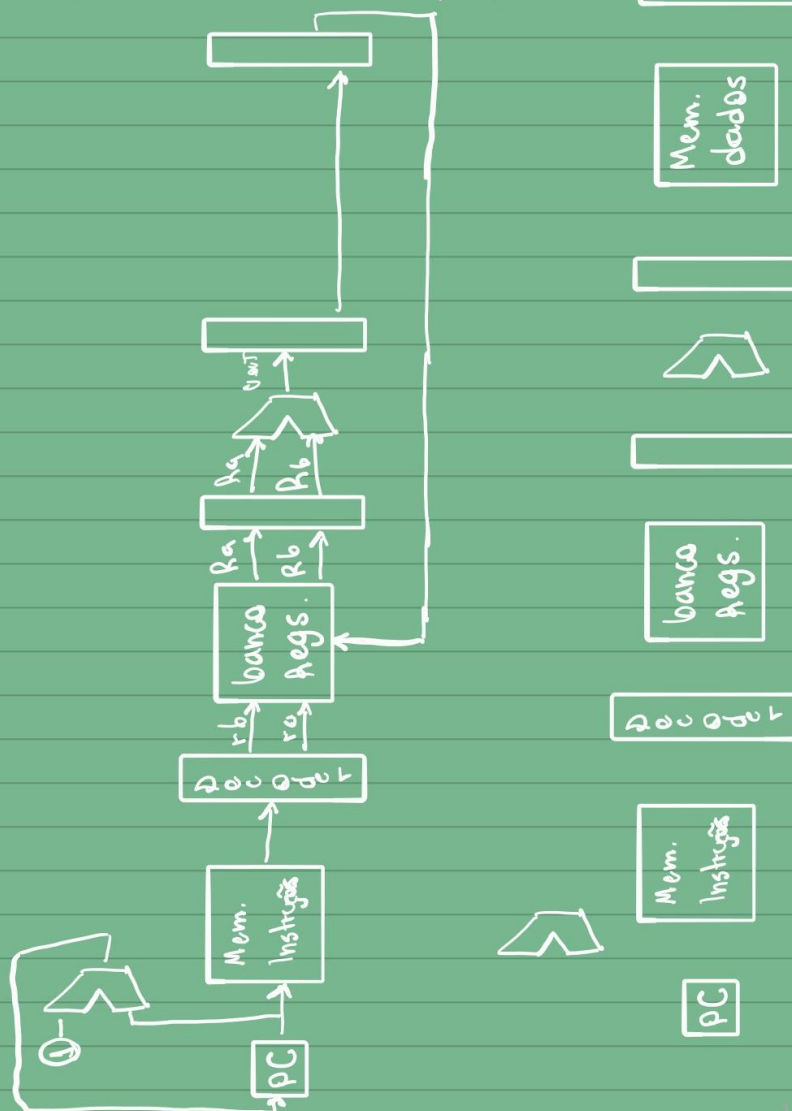
Sagui Vetorial

Diagrama do processador

DATAPATH'S

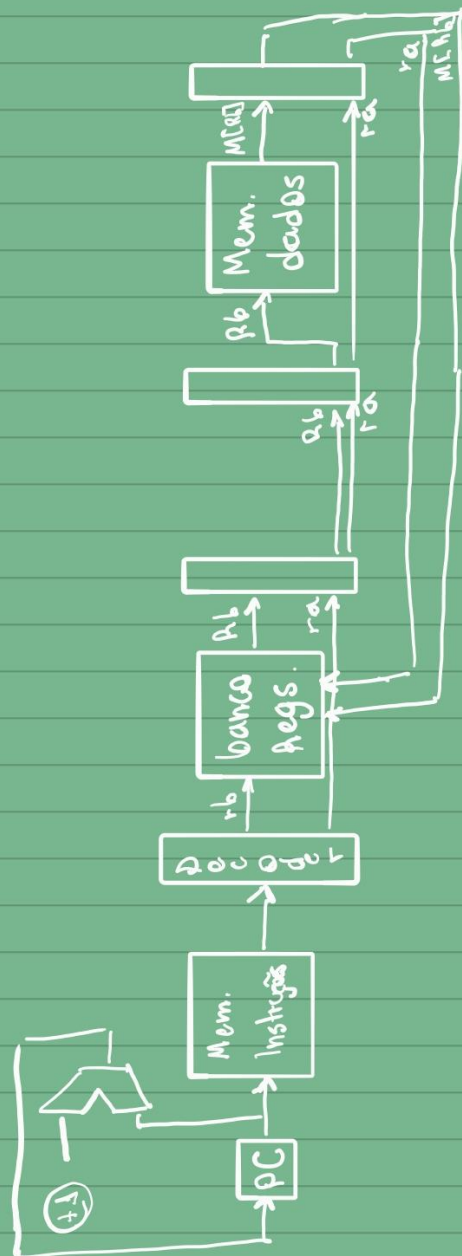
• ESCALARES

→ Tipo R / aritméticas (add, sub, and)



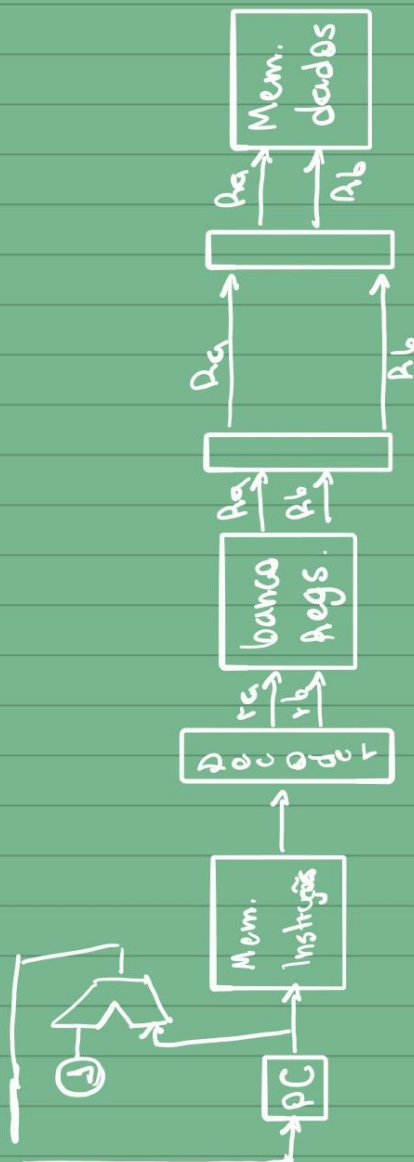
→ load

→ load



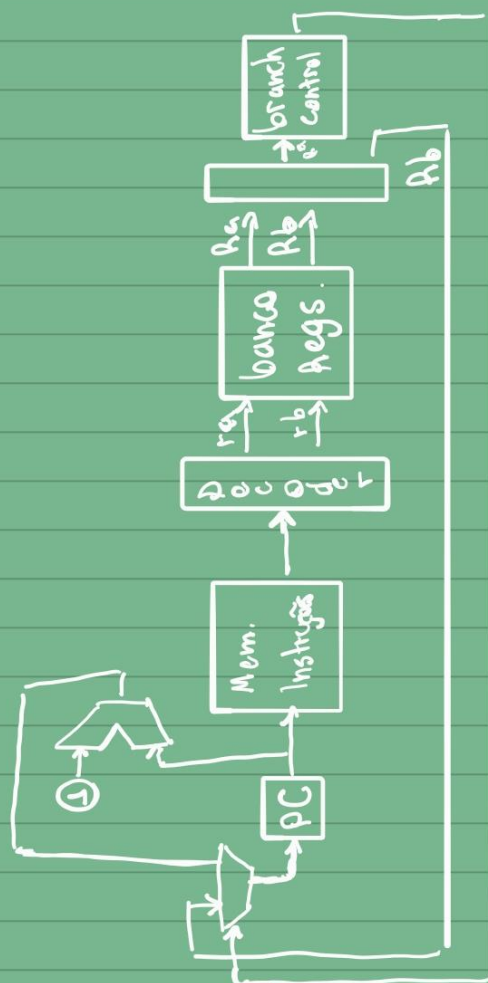
→ store

→ store



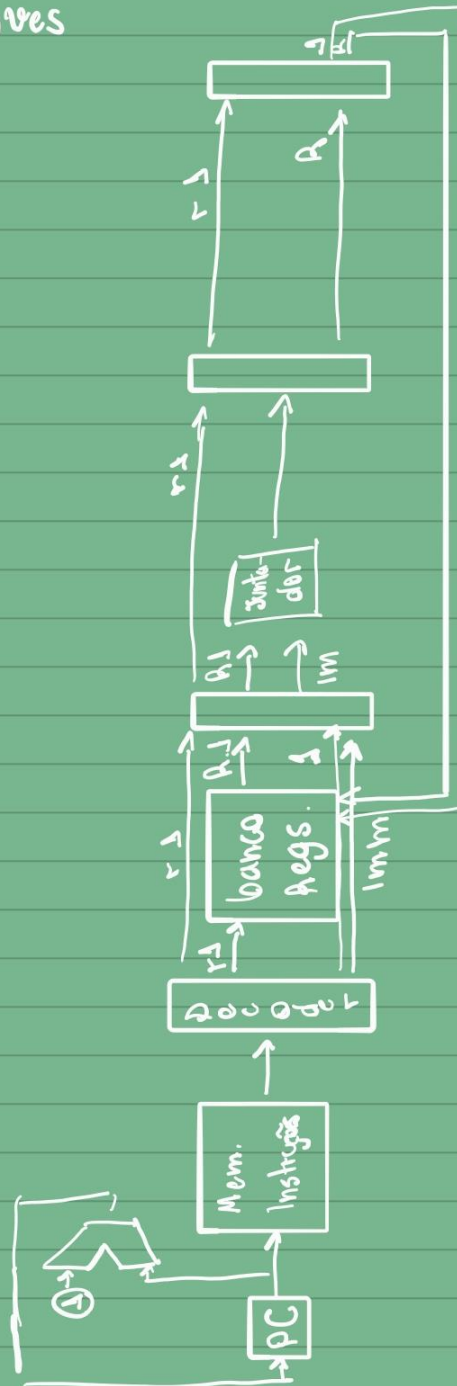
→ b735

→ br zr



→ Tipo I - Moves

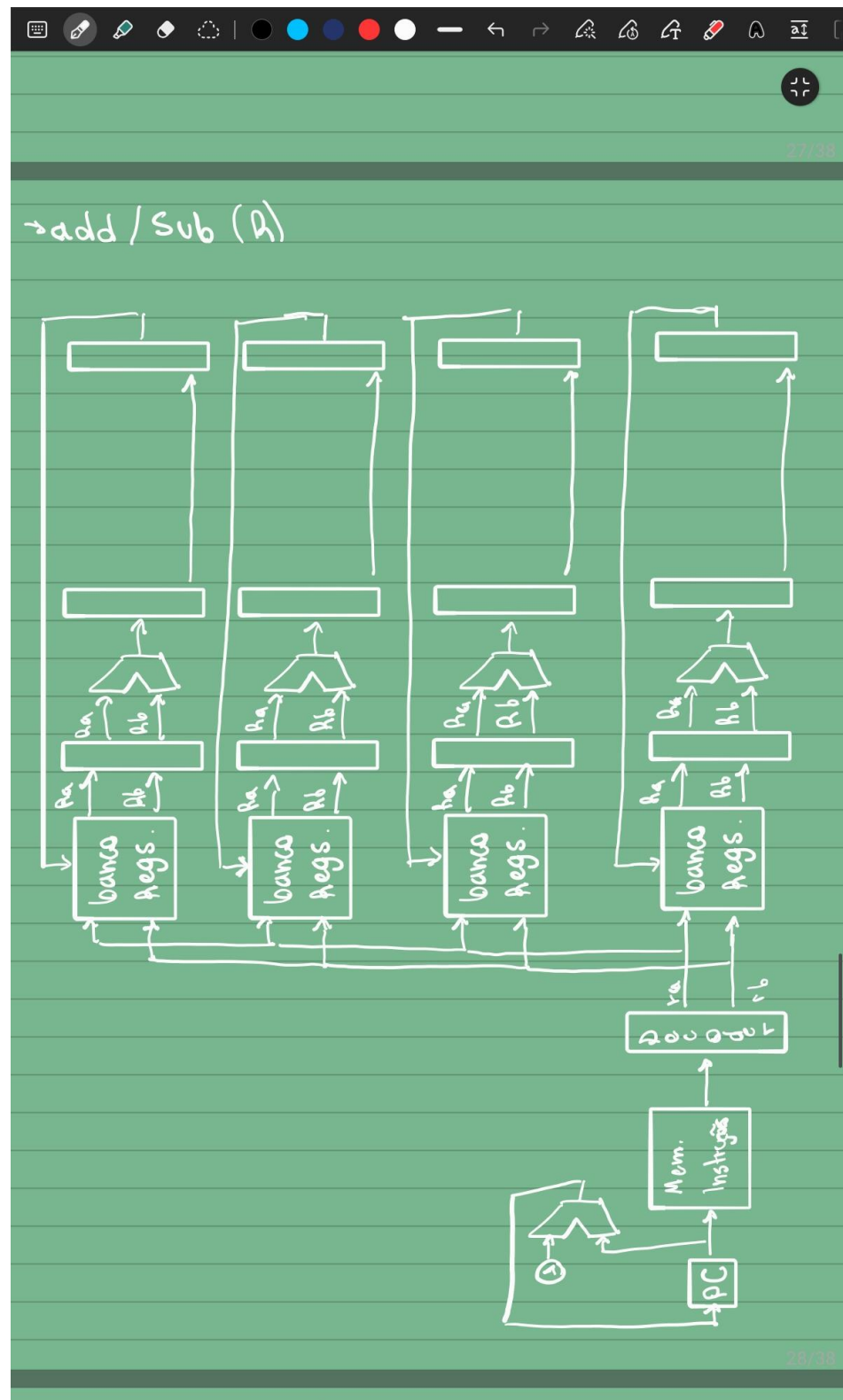
→ Tipo I - Moves

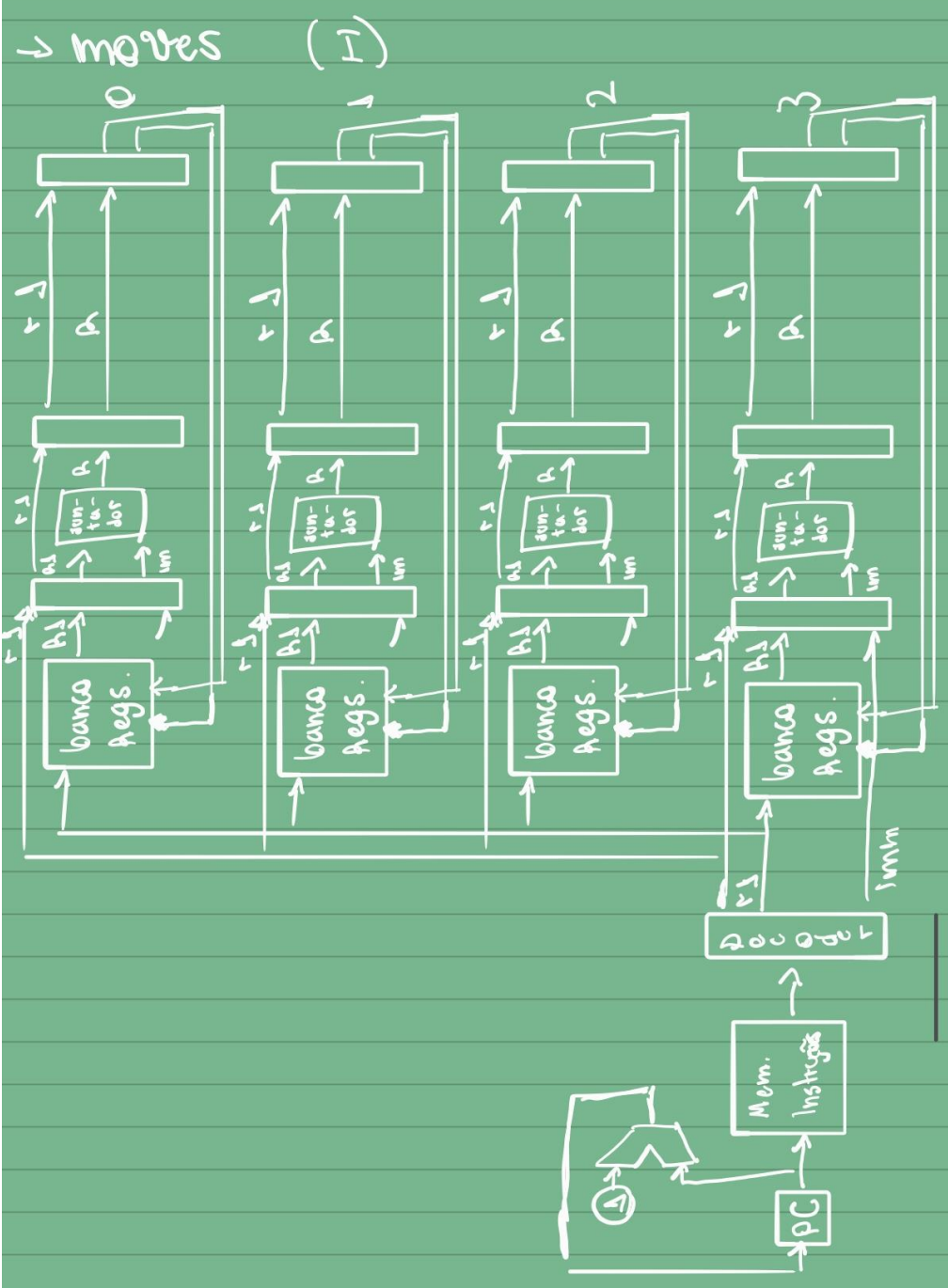


III A escalon

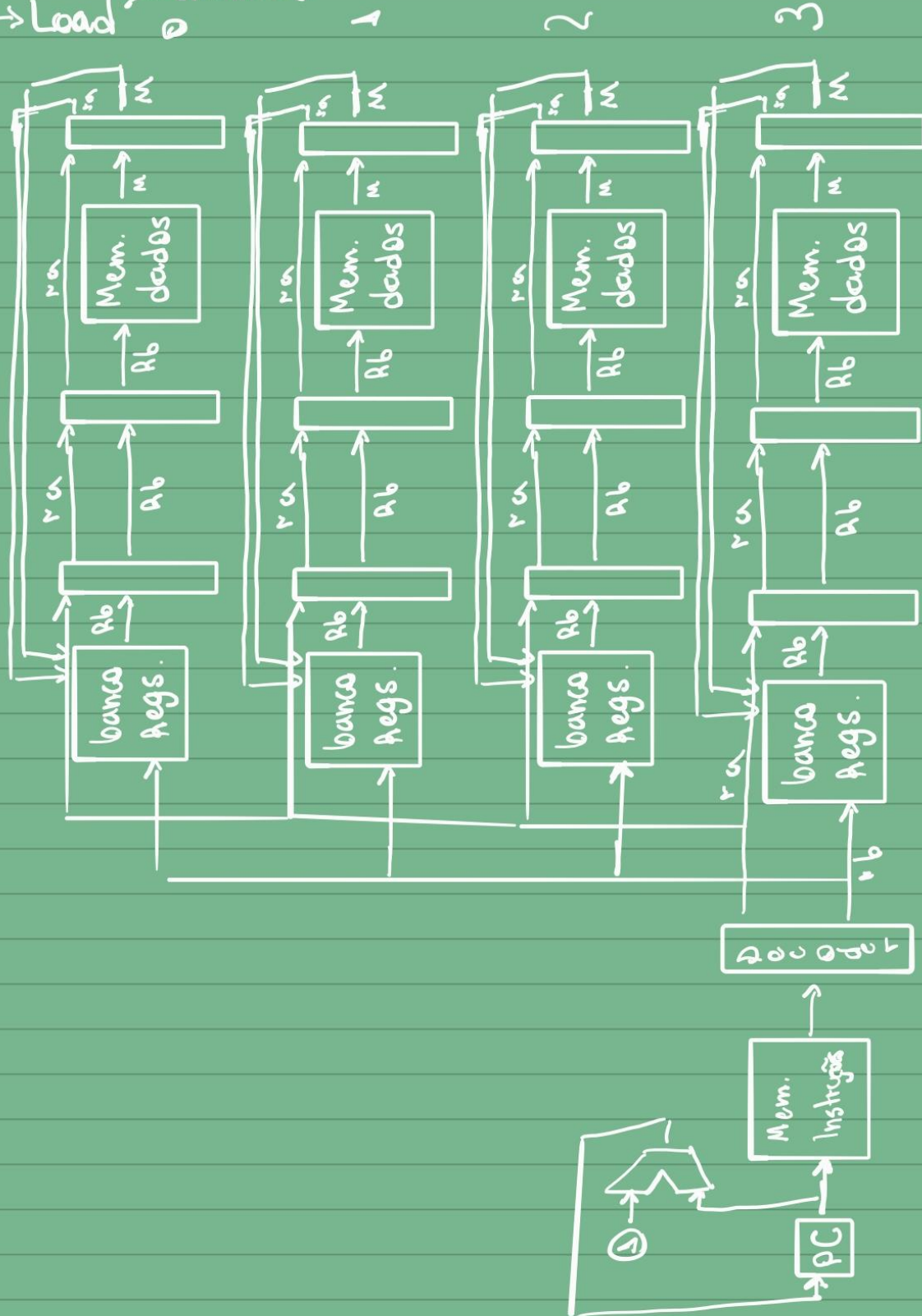
00 - add

Vetoriais

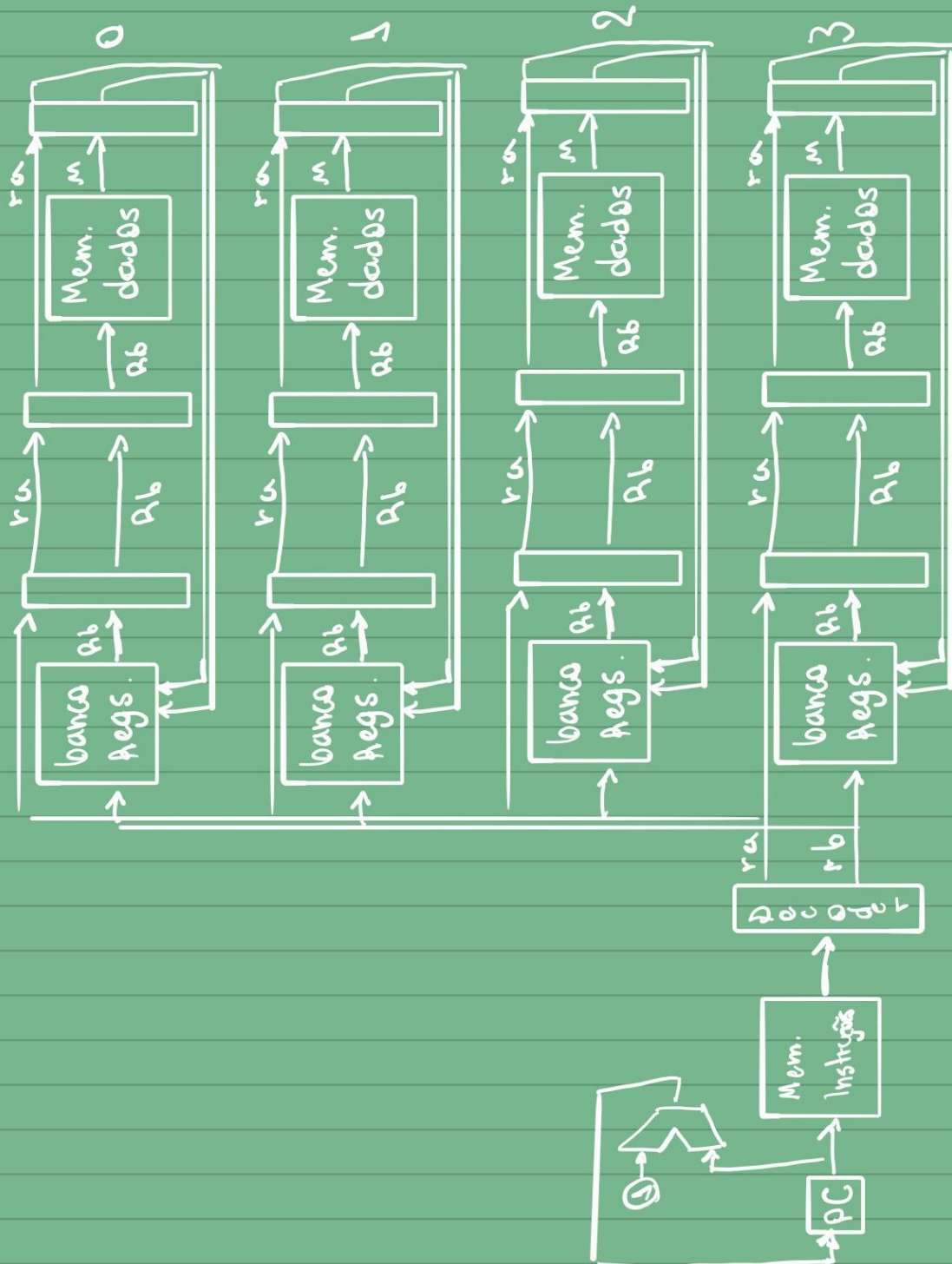




→ Load → aligned



→ Store

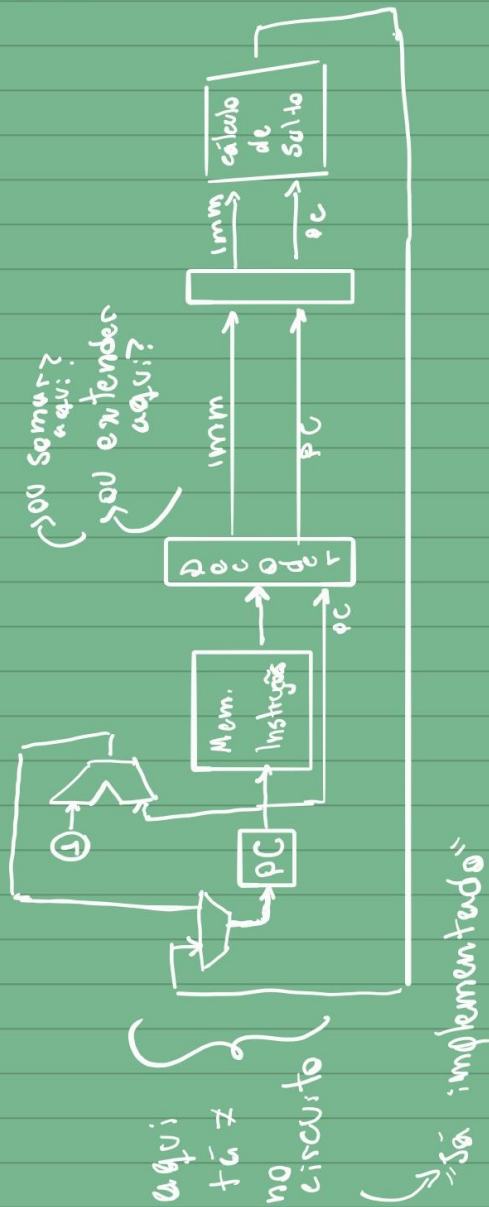


25/38

→ Como será esse load/store c/ 4 RAMs?
(→ desalinhada)

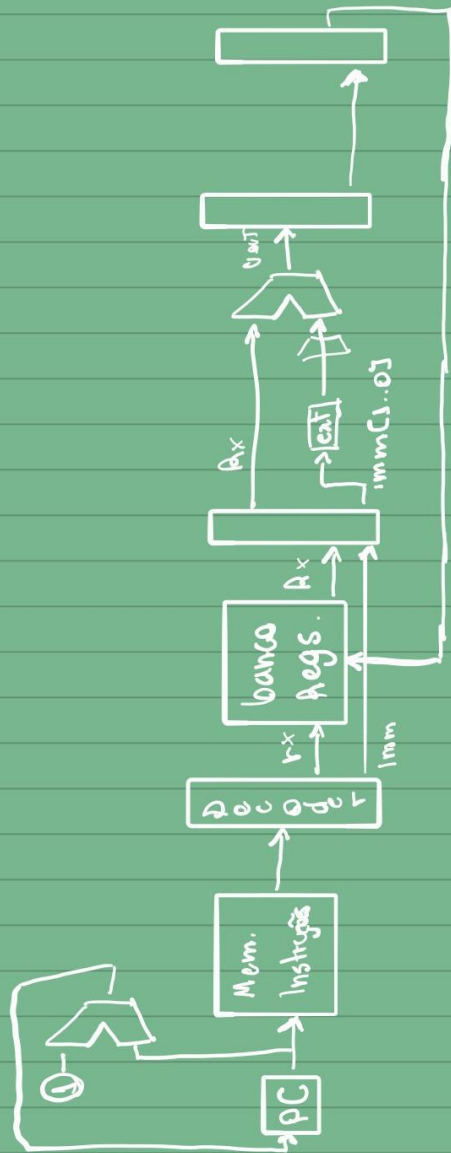
• INSTRUÇÕES NOVAS

→ S.S; imm



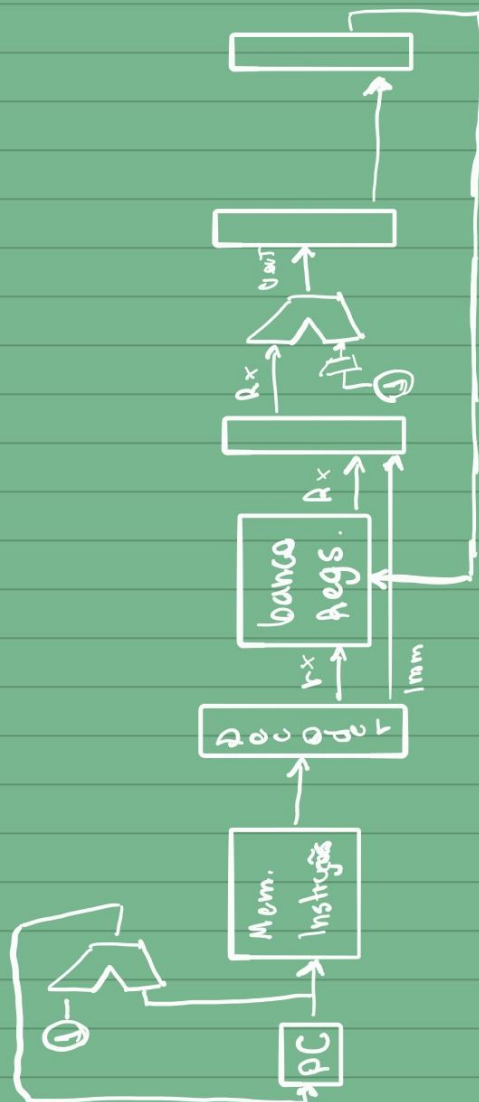
→ S.addi ra, imm

→ s.addi ra,imm

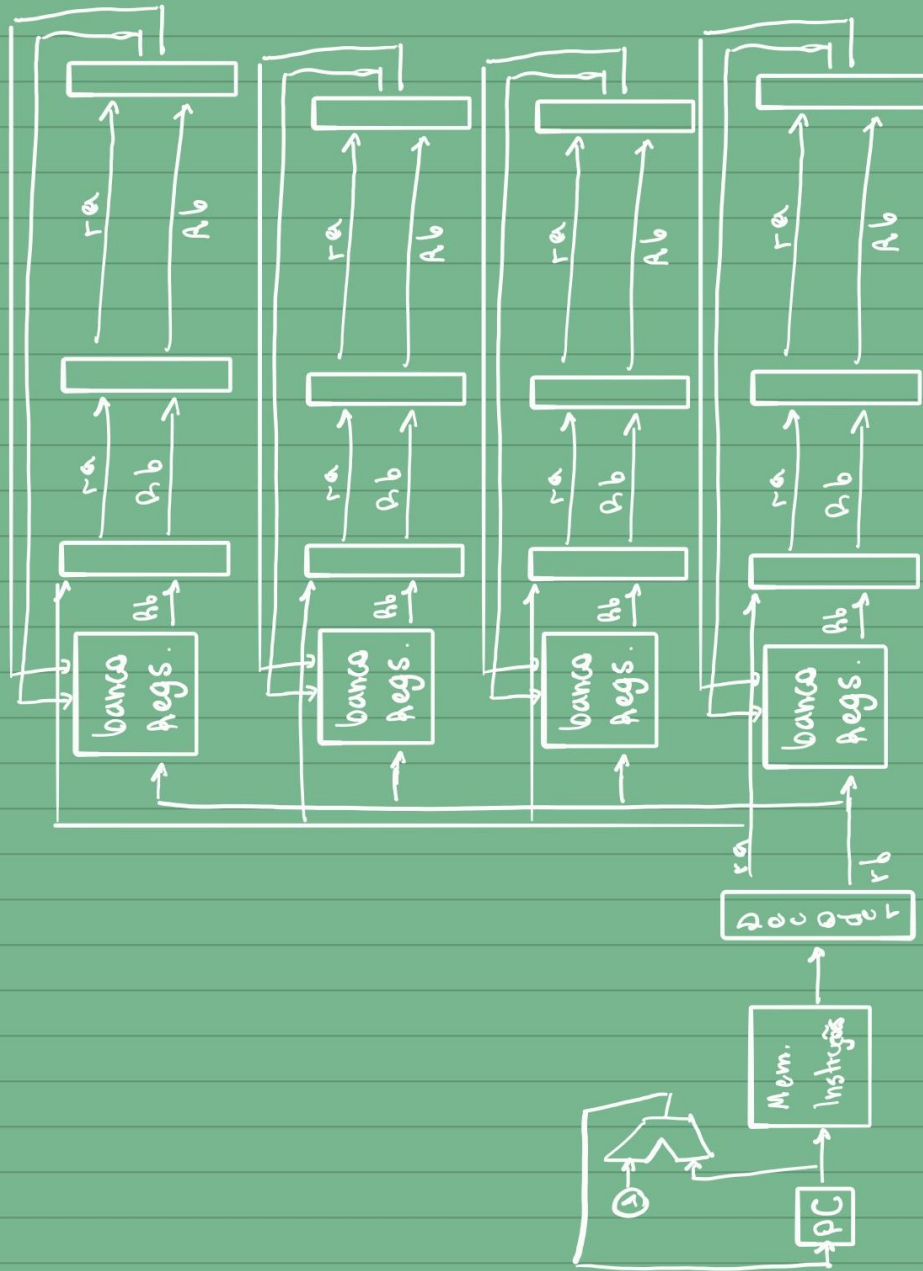


→ s.dec ra

→ s.dec ra



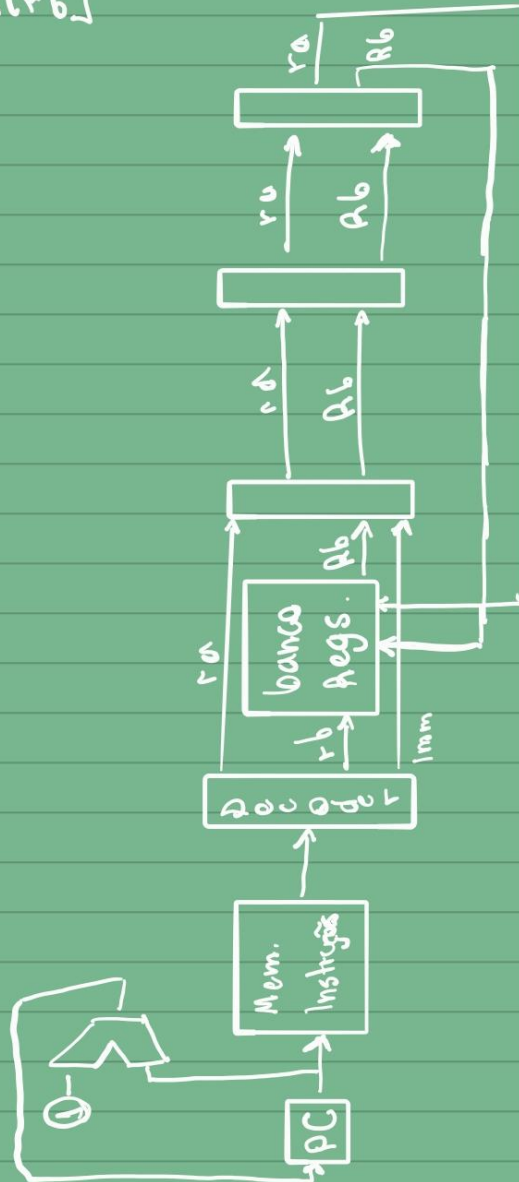
→ v. mgt ra rb



→ Construção do programa
 vA0 vA1 vA2 vA3

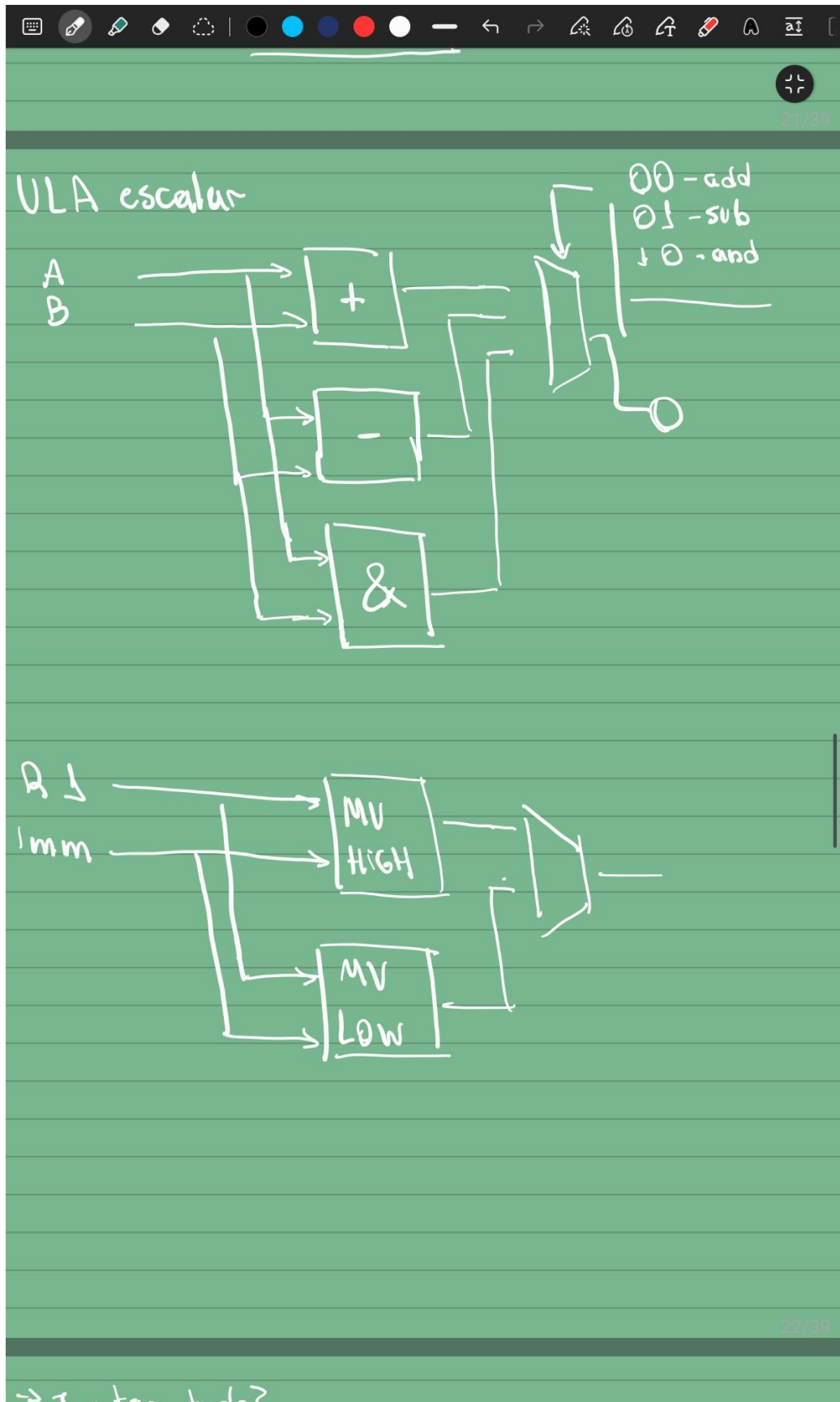
→ 5. mvr ra rb

$R[ra] \leftarrow R[r_b]$



→ 6. mvr ra rb

Projeto da ULA



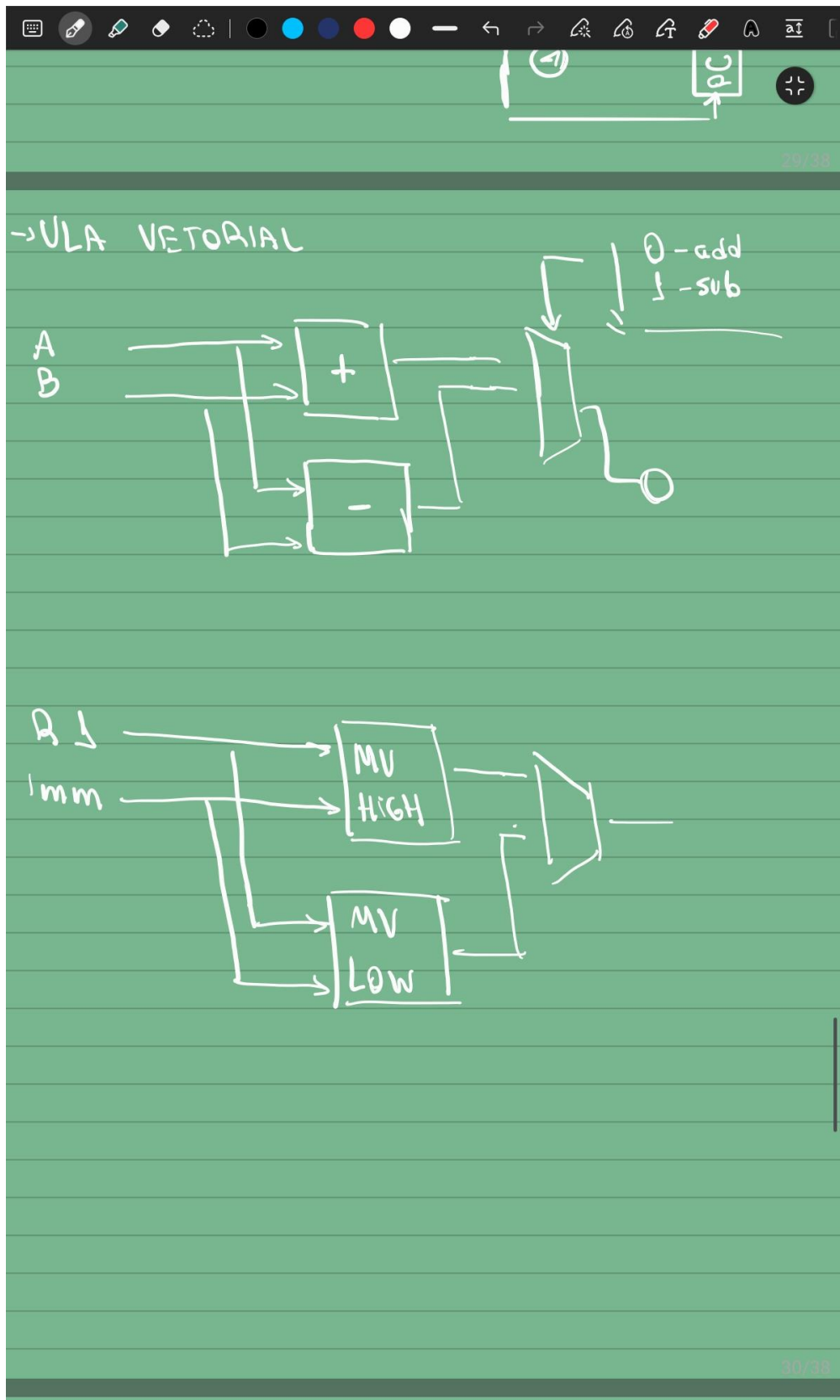


Tabela de sinais de controle

- 1 0 Recolher linhas duplicadas Mostrar todas as linhas										
16 das linhas de 16 mostradas										
Opcode[3..0]	AluOp[1..0]	MUX_RegIn[1..0]	RegWrite	MemWrite	Type_Inst	PE	Branch	LI	Move	
0000	--	01	1	0	0	0	0	0	-	
0001	--	--	0	1	0	0	0	0	-	
0010	--	10	1	0	1	0	0	0	1	
0011	--	10	1	0	1	0	0	0	0	
0100	00	00	1	0	0	0	0	0	-	
0101	01	00	1	0	0	0	0	0	-	
0110	10	00	1	0	0	0	0	0	-	
0111	--	--	0	0	0	0	1	0	-	
1000	--	01	1	0	0	1	0	0	-	
1001	--	--	0	1	0	1	0	0	-	
1010	--	10	1	0	1	1	0	0	1	
1011	--	10	1	0	1	1	0	0	0	
1100	00	00	1	0	0	1	0	0	-	
1101	01	00	1	0	0	1	0	0	-	
1110	--	11	1	0	0	0	0	0	-	
1111	00	00	1	0	0	0	0	1	-	

Assembly

Instruções novas

- s.mvr sra srb
 - Opcode: 1110
 - Tipo: R
 - Nome: move registers
 - Operação: $SR[ra] = SR[rb]$
 - Motivo: Economiza várias operações de s.sub srx, srx, que preparam o registrador para a atribuição de um valor vindo de outro registrador com a operação s.add (geralmente se usa o sr1 para montar o valor para logo em seguida atribuir a outro). Economiza 6 instruções no total.
- s.addli ra imm
 - Opcode: 1111
 - Tipo: “R”
 - $[3..2] \leftrightarrow ra$
 - $[1..0] \leftrightarrow$ imediato de 2 bits em complemento de 2
 - Nome: Add lower immediate in register
 - Operação: $SR[ra] = SR[ra] + imm$ (2 bits)
 - Motivo: É o “dec” e o “inc” incorporado em 1 instrução só (portanto, mais universal). Como o programa possui 2 loops, a inserção dessa instrução facilita muito o código pois, além de “livrar” um registrador para ele ser usado para outra coisa e excluir indiretamente algumas instruções (pode-se guardar mais um endereço em um registrador durante os loops, por exemplo), ele diminui diretamente o número de instruções para decrementar o iterador do loop. Economiza 4 instruções no total.

Código:

Escreva um programa de teste para cada processador:

```

# Uma soma de 2 vetores teste de 12 posições fazendo R=A+B. Seu código deve inicializar os
# vetores A, B e R. Os vetores devem estar alinhados na memória.
# Você deve utilizar pelo menos um loop para implementar a soma ou para inicializar o vetor
# (você deve mostrar em algum ponto que sabe trabalhar com laços).
# R deverá iniciar com zero.
# A = {0,1,2,3,4,5,6,7,8,9,10,11}.
# B={20,21,22,23,24,25,26,27,28,29,30,31}.
# No caso do processador vetorial os dados estarão distribuídos nas memórias dos PEs

```

#DECISÕES (D)

```

# 1: Antes de usar, todos os registradores foram zerados antes
# 2: Mesmo que eu saiba que o dado está zerado, eu vou zerá-lo de novo por precaução
# 3: Considerei que vr2 chega com {12, 13, 14, 15} e vr1 com {4, 4, 4, 4} eem inializa_2
# 4: Considerei que os endereços chegam certos em inicializa_2 para a inicialização do segundo vetor
# 5: Considerei que o vr1 se manterá com constante 4 na inicialização dos 2 vetores
# 6: Não desviarei para labels que estão abaixo da label atual
# 7: Load/Store não mexem no endereço de acesso a memória

```

```

#NOVAS INSTRUÇÕES: s.addli ra, limm e s.mvr sra srb

```

```

main:

```

```

#"Inicializa_1"

```

```

# Inicializa iterador -----

```

```

# Inicializa "n" (número de iterações) em sr3

```

```

s.movh 0 #D1

```

```

s.movl 3

```

```

s.mvr sr3, sr1 # sr3 = 3

```

```

# Inicializa endereços de jump -----

```

sr2 recebe endereço do inicializa_2

s.movh 0

s.movl -1

s.mvr sr2, sr1

sr1 recebe endereço de inicializa_vetores

s.movh 1

s.movl -7

Inicializa dados iniciais -----

v.sub vr2, vr2 # vr2 = {0, 0, 0, 0} - D1

v.add vr2, vr0 # vr2 = {0, 1, 2, 3} - dados iniciais

Inicializa endereços iniciais -----

v.sub vr3, vr3 # vr3 = {0, 0, 0, 0} - D1

v.add vr3, vr0 # vr3 = {0, 1, 2, 3} - endereços iniciais

Define vr1 com a constante 4 -----

v.movh 0 #D1

v.movl 4

Desvia para "inicializa_vetores"

s.brzr sr0, sr1

inicializa_2:

Inicializa iterador -----

Inicializa "n" (número de iterações) em sr3

s.movh 0 # D2

s.movl 3

s.mvr sr3, sr1 # sr3 = 3

Inicializa endereços de jump -----

sr2 recebe endereço do próximo label - soma_setup

s.movh 1

s.movl -1

s.mvr sr2, sr1

sr1 recebe endereço de inicializa_vetores (loop)

s.movh 1

s.movl -7

D3

v.add vr2, vr1 # vr2 = {16, 17, 18, 19}

v.add vr2, vr1 # vr2 = {20, 21, 22, 23}

D4

D5

D6

Ao desviar para cá, tem que ter o endereço em VR3, dados em VR2 e 4 em VR1

inicializa_vetores:

v.st vr2, vr3 # Guarda dados na memória

v.add vr2, vr1 # vr2 = vr2 + vr1 / vr2 += 4 -> itera dados - estão em sequência

v.add vr3, vr1 # vr3 = vr3 + vr1 / vr3 += 4 -> itera endereço

```

# "if (i == 3) finaliza loop"
s.addli sr3, -1 # n--

s.brzr sr3, sr2 # Se sr3 == 0 -> pula para próximo label

s.brzr sr0, sr1 # Desvia para "inicializa_vetores"

soma_setup:
# Inicializa iterador

# Inicializa "n" (número de iterações) em sr3
s.movh 0 # D2
s.movl 3
s.mvr sr3, sr1 # sr3 = 3

# Inicializa endereço de fim
s.movh 3 # sr1 = &fim
s.movl -7
s.mvr sr2, sr1 # sr2 = sr1

# Monta endereço de soma_vetores
s.movh 2
s.movl -7

# Inicializa endereços iniciais
v.sub vr3, vr3 # vr3 = {0, 0, 0, 0}
v.add vr3, vr0 # vr3 = {0, 1, 2, 3}

soma_vetores:

```

v.ld vr2, vr3 # Carrega 1/3 do vetor 1 em vr2

Monta o endereço do vetor 2 em vr3

v.movh 0 # vr1 = 12

v.movl -4 # 12 em complemento de 2

v.add vr3, vr1 # vr3 = endereço vetor 2

v.ld vr1, vr3 # Carrega 1/3 do vetor 2 em vr1

v.add vr2, vr1 # Soma vetor 1 e 2 e armazena em vr2

Monta endereço do vetor resultado em vr3

v.movh 0

v.movl -4

v.add vr3, vr1

v.st vr2, vr3 # Guarda vetor resultado

"if (i == 3) finaliza programa"

s.addli sr3, -1 # n--

s.brzr sr3, sr2

Se não...

Monta próximo endereço base

v.movh 1

v.movl 4

v.sub vr3, vr1 #vr3 = vr3 - 20

#{diminuiu 24 endereços para voltar ao vetor 1 e soma 4 para ir para o próximo bloco)

```
s.brzr sr0 sr1 # Desvia para soma_vetores
```

```
fim:
```

```
# Loop infinito
```

```
s.movh 3
```

```
s.movl -7
```

```
s.brzr sr0, sr1
```

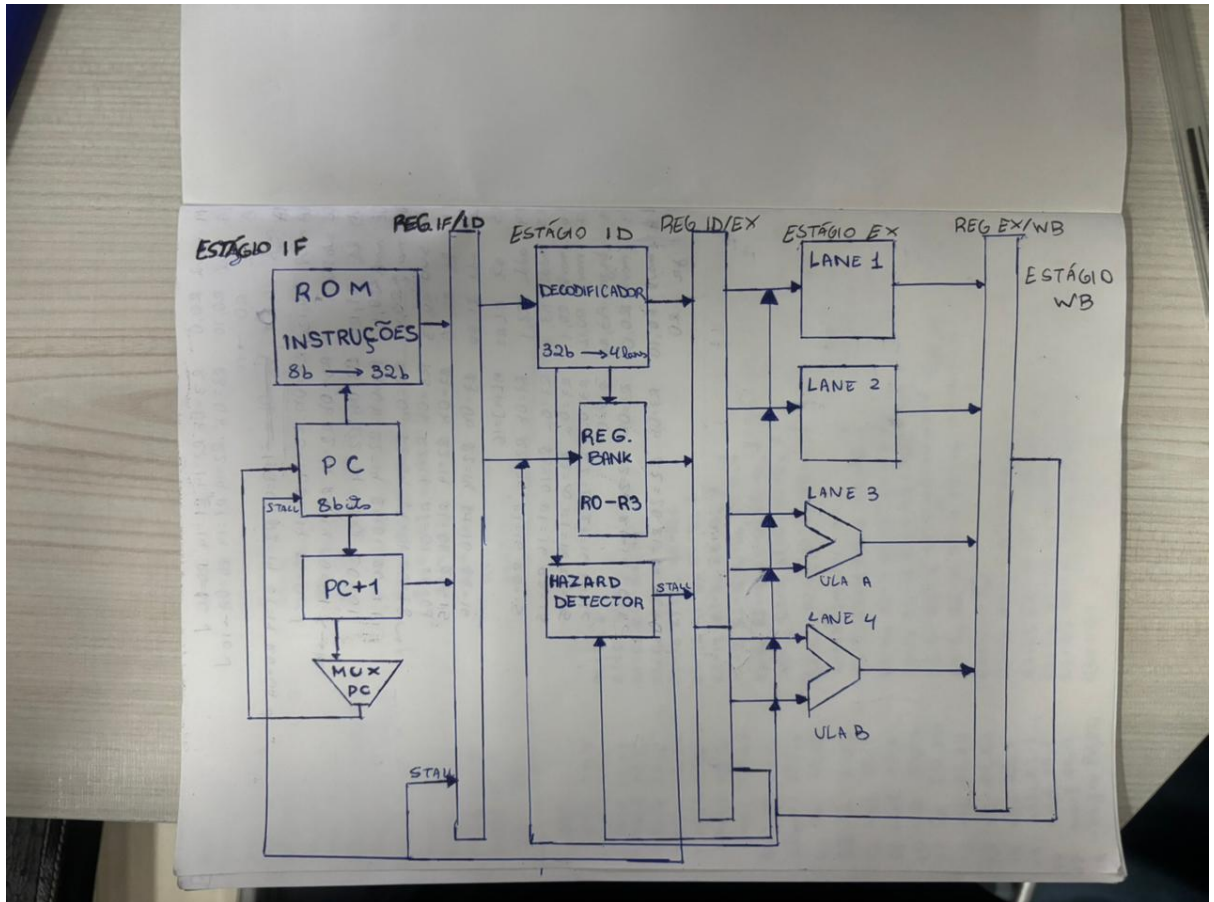
Sagui VLIW

1. Descrição da arquitetura

1.1 Organização do processador

- 4 lanes
 - Lane 1 (LD/ ST/ MOV)
 - Lane 2 (BR/ JMP)
 - Lane 3 (ULA A)
 - Lane 4 (ULA B)
- 5 estágios
 - **IF (Instruction Fetch)**: busca da instrução na memória ROM
 - **ID (Instruction Decode)**: decodificação da instrução e leitura dos operandos no banco de registradores
 - **EX (Execute)**: execução das operações aritméticas, lógicas e cálculo de endereços
 - **MEM (Memory Access)**: acesso à memória de dados para instruções de carga e armazenamento
 - **WB (Write Back)**: escrita dos resultados no banco de registradores
- Banco de registradores com duas portas de escrita simultâneas e múltiplas portas de leitura para atender às necessidades das quatro lanes. Lane 3 e 4 dividem a mesma porta de escrita, sendo a lane 3 priorizada caso queiram escrever ao mesmo tempo.
- Implementado mecanismo de **forwarding**.
- Implementada unidade de detecção de hazards, responsável pela inserção de **stalls** quando forwarding não é suficiente.

- Suporte para instruções de desvio e salto, quando isso ocorre um mecanismo de **flush** é utilizado para invalidar instruções incorretamente propagadas pelo pipeline.
- Instruções são armazenadas em uma memória ROM, enquanto os dados em uma memória RAM, caracterizando uma **organização do tipo Harvard**.
- Arquitetura validada por meio da execução de um programa de soma vetorial, que inicia dois vetores na memória e calcula o terceiro contendo a soma de cada elemento correspondente dos vetores.
- Datapath desenhado:



1.2 Estágios do pipeline

IF – Instruction Fetch

Nesse estágio a instrução é buscada da memória ROM utilizando o valor atual do Program Counter (PC). A instrução e o valor do PC são armazenados no registrador de pipeline IF/ID.

ID – Instruction Decode

Nesse estágio ocorre a decodificação das instruções das quatro lanes, a geração dos sinais de controle e a leitura dos operandos no banco de registradores. Também são realizadas verificações de hazards e gerações do sinal de stall.

EX – Execute/ MEM – Memory Access

Nesse estágio são executadas operações aritméticas, lógicas e de controle. As ULAs realizam operações aritméticas e cálculos de endereços de acesso à memória. O estágio MEM recebe os dados retornados pela RAM e os propaga para o estágio WB.

WB – Write Back

Nesse estágio ocorre a escrita dos resultados no banco de registradores.

Registradores de Pipeline

- IF/ ID
- ID/ EX
- EX / WB

Tratamento de hazards

- **Forwarding**
 - Não existe uma unidade de forwarding, mas o mecanismo é o mesmo, ocorre a comparação do “resultado antigo” com o resultado novo e se é escolhido o que deve ser passado.
 - Essa comparação ocorre internamente nas lanes, mas externamente no banco de registradores
- **Stall**
 - Nem todas as dependências puderam ser resolvidas por forwarding, principalmente na instrução de load. Nesse caso, o Hazard_Detector gera um stall que interrompe temporariamente o avanço do pipeline.
- **Flush**
 - Ocorre em instruções de desvio e salto, em que instruções “inválidas” acabam passando pelo pipeline. Quando um brranch ou jump altera o fluxo de execução, o sinal de flush é gerado e invalida as instruções carregadas incorretamente, substituindo-as por NOPs.

1.3 Formato das instruções

Lane 1 – LD/ ST/ MOV

opcode	mnemônico	mux_regin	l1regwrite	memwrite	memread	mux_imm	sel_lane[1:0]
0100	ld	0	1	0	1	-	00
0101	st	0	0	1	0	-	--
0110	movh	1	1	0	0	0	01
0111	movl	1	1	0	0	1	01
1010	mov	1	1	0	0	-	10
1111	nop	0	0	0	0	-	--

Lane 2 – BR/ JMP

Opcode	Mnemônico	branch[3]	jr[2]	relative[1]	contidion[0]
0000	brzr	1	0	0	0
0001	brzi	1	0	1	1
0010	jr	0	1	0	-
1111	nop	0	0	-	-

Lane 3 e 4 – ULA A e ULA B

Opcode	Mnemônico	aluop[4:2]	regwrite[1]	mux_ula_b[0]
1000	add	000	1	0
1001	sub	001	1	0
1011	or	010	1	0
1100	not	011	1	0
1101	slr	100	1	0
1110	srr	101	1	0
0011	addi	000	1	1
1111	nop	111	0	-

Tabela Geral

Opcode	Mnemônico	Lane
0000	brzr	2
0001	brzi	2
0010	jr	2
0011	addi	3/4
0100	ld	1
0101	st	1
0110	movh	1
0111	movl	1
1000	add	3/4
1001	sub	3/4
1010	mov	1
1011	or	3/4
1100	not	3/4
1101	slr	3/4
1110	srr	3/4
1111	nop	-

Detector Tipo I – Lane 1

Opcode	Mnemônico	Tipo_I_L1
0110	movh	1
0111	movl	1
demais	-	0

Detector Tipo I – Lane 2

Opcode	Mnemônico	Tipo_I_L2
0001	brzi	1
demais	-	0

Detector Tipo I – Lanes 3 e 4

Opcode	Mnemônico	Tipo_I_L3/L4
0011	addi	1
demais	-	0

1.4 Banco de registradores

- 4 registradores de propósito geral: R0, R1, R2 e R3. Cada registrador possui 8 bits de largura
- Política Write-First: O banco de registradores foi implementado utilizando a política write-first. Assim, quando uma operação de leitura e uma operação de escrita ocorre

simultaneamente sobre o mesmo registrador durante um ciclo de clock, o valor retornado corresponde ao dado que está sendo escrito no ciclo.

1.5 Memória

Organização dos Dados

Durante os testes foi utilizada a seguinte organização da memória:

Endereços	Conteúdo
0 – 11	Vetor A
12 – 23	Vetor B
24 – 35	Vetor Resultado (R)

Após a execução do programa, os vetores armazenavam:

Vetor A: valores de 0 a 11.

Vetor B: valores de 20 a 31.

Vetor R: soma elemento a elemento dos vetores A e B.

1.6 Unidades Funcionais

Program Counter (PC)

Responsável por armazenar o endereço da próxima instrução a ser executada. O valor do PC é atualizado sequencialmente ou modificado por instruções de desvio e salto.

Memória de Instruções (InstMem)

Armazena o programa executado pelo processador. Durante o estágio IF, a instrução localizada no endereço indicado pelo PC é lida e enviada para o pipeline.

Registrador IF/ID

Armazena temporariamente a instrução buscada e as informações necessárias para o estágio de decodificação.

Decodificador

Responsável por interpretar os campos da instrução e gerar os sinais de controle necessários para os demais componentes do processador.

Hazard Detector

Monitora dependências entre instruções presentes no pipeline. Quando uma situação de risco é detectada, gera sinais de stall e flush para garantir a execução correta do programa.

Banco de Registradores (Register Bank)

Armazena os operandos e resultados utilizados pelas instruções. Possui múltiplas portas de leitura e duas portas de escrita, permitindo a operação da arquitetura VLIW implementada.

Registrador ID/EX

Armazena operandos, sinais de controle e demais informações produzidas durante a decodificação para utilização no estágio de execução.

Extensão de Registradores ID/EX

Bloco responsável por transportar e organizar os sinais e operandos necessários para as unidades de execução presentes no estágio EX.

Lane 1

Unidade responsável pela execução das instruções de acesso à memória, incluindo operações de leitura (LD) e escrita (ST).

Lane 2

Unidade responsável pela execução das instruções de controle de fluxo, como desvios condicionais (BRZR) e saltos (JR).

ULA A

Executa operações aritméticas e lógicas associadas à terceira lane do processador, incluindo instruções como ADD, SUB, ADDI, MOVH e MOVL.

ULA B

Executa operações aritméticas e lógicas associadas à quarta lane do processador.

Registrador EX/WB

Armazena os resultados produzidos pelas unidades de execução antes da etapa de escrita no banco de registradores.

2. Código Assembly – Soma de Vetores $R = A + B$

Endereços de memória de dados: A = 0x00 a 0x0B, B = 0x0C a 0x17, R = 0x18 a 0x23

Formato: [Lane1 LD/ST/MOV | Lane2 BR/JMP | Lane3 ULA A | Lane4 ULA B]

```
; =====  
; FASE 1 — Inicializa R[24..35] com zeros  
; =====  
  
; end 0  
[ movh R0, 1 | nop | nop | nop ]  
; end 1 R0=24  
[ movl R0, 8 | nop | nop | nop ]
```

```

; end 2 R2=24
[ mov R2, R0 | nop | nop | nop ]

; end 3
[ movh R0, 0 | nop | nop | nop ]
; end 4 R0=0
[ movl R0, 0 | nop | nop | nop ]
; end 5 R1=0
[ mov R1, R0 | nop | nop | nop ]

; end 6
[ movh R0, 0 | nop | nop | nop ]
; end 7 R0=12
[ movl R0, 12 | nop | nop | nop ]
; end 8 R3=12
[ mov R3, R0 | nop | nop | nop ]

; init_R = end 9
; end 9
[ st R1, R2 | nop | addi R3,-1 | nop ]
; end 10
[ nop | nop | addi R2,1 | nop ]
; end 11
[ movh R0,1 | nop | nop | nop ]
; end 12
[ movl R0,1 | nop | nop | nop ]
; end 13
[ nop | brzr R3,R0 | nop | nop ]
; end 14
[ movh R0,0 | nop | nop | nop ]
; end 15
[ movl R0,9 | nop | nop | nop ]
; end 16
[ nop | jr R0 | nop | nop ]

; fim_R = end 17

; =====
; FASE 2 — Inicializa A[0..11] = 0..11
; =====

; end 17
[ movh R0,0 | nop | nop | nop ]
; end 18
[ movl R0,0 | nop | nop | nop ]
; end 19
[ mov R2,R0 | nop | nop | nop ]
; end 20
[ mov R1,R0 | nop | nop | nop ]

; end 21
[ movh R0,0 | nop | nop | nop ]
; end 22
[ movl R0,12 | nop | nop | nop ]
; end 23
[ mov R3,R0 | nop | nop | nop ]

; init_A = end 24
; end 24
[ st R1,R2 | nop | addi R3,-1 | nop ]
; end 25

```

```

[ nop | nop | addi R2,1 | nop ]
; end 26
[ nop | nop | nop | addi R1,1 ]

; end 27
[ movh R0,2 | nop | nop | nop ]
; end 28
[ movl R0,1 | nop | nop | nop ]
; end 29
[ nop | brzr R3,R0 | nop | nop ]

; end 30
[ movh R0,1 | nop | nop | nop ]
; end 31
[ movl R0,8 | nop | nop | nop ]
; end 32
[ nop | jr R0 | nop | nop ]

; fim_A = end 33

```

```

;
;          FASE          3          —          Inicializa          B[12..23]          =          20..31
;=====

;
;          movh          R0,0          |          nop          |          nop          |          nop          ]          33
;
;          movl          R0,12         |          nop          |          nop          |          nop          ]          34
;
;          mov R2,R0 | nop | nop | nop ]          35

;
;          movh          R0,1          |          nop          |          nop          |          nop          ]          36
;
;          movl          R0,4          |          nop          |          nop          |          nop          ]          37
;
;          mov R1,R0 | nop | nop | nop ]          38

;
;          movh          R0,0          |          nop          |          nop          |          nop          ]          39
;
;          movl          R0,12         |          nop          |          nop          |          nop          ]          40
;
;          mov R3,R0 | nop | nop | nop ]          41

;          init_B          =          end          42
;
;          st          R1,R2          |          nop          |          addi          R3,-1          |          nop          ]          42
;
;          nop          |          nop          |          addi          R2,1          |          nop          ]          43
;
;          [ nop | nop | nop | addi R1,1 ]          44

;
;          movh          R0,3          |          nop          |          nop          |          nop          ]          45
;
;          movl          R0,3          |          nop          |          nop          |          nop          ]          46
;
;          [ nop | brzr R3,R0 | nop | nop ]          47

```

```

; end 48
[ movh R0,2 | nop | nop | nop ]
; end 49
[ movl R0,10 | nop | nop | nop ]
; end 50
[ nop | jr R0 | nop | nop ]

```

```

; fim_B = end 51

```

```

;
;          FASE          4          —          Soma          Vetorial
;          R[i]          =          A[i]          +          B[i]
; =====

```

```

;
;          movh          R0,0          |          nop          |          nop          |          nop          ]          51
;
;          movl          R0,12         |          nop          |          nop          |          nop          ]          52
;
;          mov R1,R0 | nop | nop | nop ]          53

```

```

;
;          movh          R0,1          |          nop          |          nop          |          nop          ]          54
;
;          movl          R0,8          |          nop          |          nop          |          nop          ]          55
;
;          mov R2,R0 | nop | nop | nop ]          56

```

```

;
;          movh          R0,0          |          nop          |          nop          |          nop          ]          57
;
;          movl          R0,12         |          nop          |          nop          |          nop          ]          58
;
;          mov R3,R0 | nop | nop | nop ]          59

```

```

; soma_vetores = end 60

```

```

;
;          movh          R0,0          |          nop          |          nop          |          nop          ]          60
;
;          movl          R0,12         |          nop          |          nop          |          nop          ]          61
;
;          end          62          endereço          de          A[i]
;          nop          |          nop          |          sub          R0,R3          |          nop          ]          63
;
;          ld R1,R0 | nop | nop | nop ]          A[i]

```

```

;
;          movh          R0,1          |          nop          |          nop          |          nop          ]          64
;
;          movl          R0,8          |          nop          |          nop          |          nop          ]          65
;
;          end          66          endereço          de          B[i]
;          nop          |          nop          |          sub          R0,R3          |          nop          ]          67
;
;          ld R0,R0 | nop | nop | nop ]          B[i]

```

```

;
;          end          68          A[i]          +          B[i]
;          nop | nop | add R1,R0 | nop ]

```

```

;
;          end          69
;          st R1,R2 | nop | nop | nop ]

```



```

;                                     end                                70
[      nop      |      nop      |      addi      R2,1      |      nop      ]
;                                     end                                71
[ nop | nop | addi R3,-1 | nop ]

;                                     end                                72
[      movh      R0,4      |      nop      |      nop      |      nop      ]
;                                     end                                73
[      movl      R0,14     |      nop      |      nop      |      nop      ]
;                                     end                                74
[ nop | brzr R3,R0 | nop | nop ]

;                                     end                                75
[      movh      R0,3      |      nop      |      nop      |      nop      ]
;                                     end                                76
[      movl      R0,12     |      nop      |      nop      |      nop      ]
;                                     end                                77
[ nop | jr R0 | nop | nop ]

;                                     fim      =      end                                78
[ nop | nop | nop | nop ]

```

3. ROM – HEX

v3.0 hex words addressed

```

00: 61ffffff 78ffffff a8ffffff 60ffffff 70ffffff a4ffffff 60ffffff 7cffffff
08: acffffff 56ff3fff ffff3935 61ffffff 71ffffff ff0cffff 60ffffff 79ffffff
10: ff20ffff 60ffffff 70ffffff a8ffffff a4ffffff 60ffffff 7cffffff acffffff
18: 56ff3fff ffff39ff ffff35ff 62ffffff 71ffffff ff0cffff 61ffffff 78ffffff
20: ff20ffff 60ffffff 7cffffff a8ffffff 61ffffff 74ffffff a4ffffff 60ffffff
28: 7cffffff acffffff 56ff3fff ffff39ff ffff35ff 63ffffff 73ffffff ff0cffff
30: 62ffffff 7affffff ff20ffff 60ffffff 7cffffff a4ffffff 61ffffff 78ffffff
38: a8ffffff 60ffffff 7cffffff acffffff 60ffffff 7cffffff ffff93ff 44ffffff
40: 61ffffff 78ffffff ffff93ff 40ffffff ffff84ff 56ffffff ffff39ff ffff3fff
48: 64ffffff 7effffff ff0cffff 63ffffff 7cffffff ff20ffff ffffffff 00000000

```

4. Resultado da execução

v3.0 hex words addressed

```

00: 00 01 02 03 04 05 06 07 08 09 0a 0b 14 15 16 17
10: 18 19 1a 1b 1c 1d 1e 1f 14 16 18 1a 1c 1e 20 22
20: 24 26 28 2a 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```